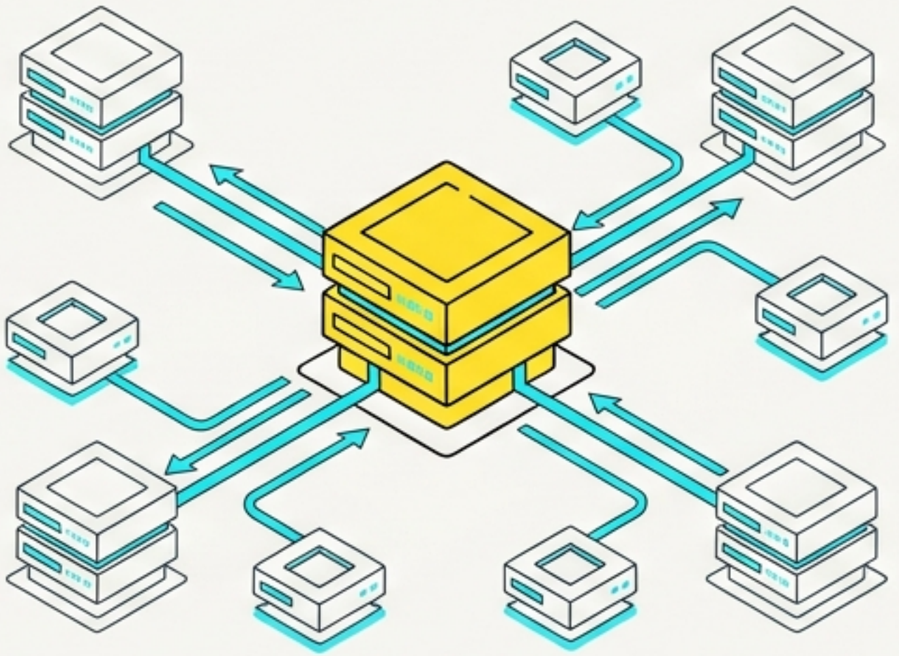
HDFS & Production Clusters

A comprehensive command reference and architectural study guide for Hadoop Distributed File System operations.

Distilled from Data Engineer Session-3



Hadoop requires five core services to be actively running.

Action

```
$ start-all.sh
```

Initializes the entire Hadoop cluster.

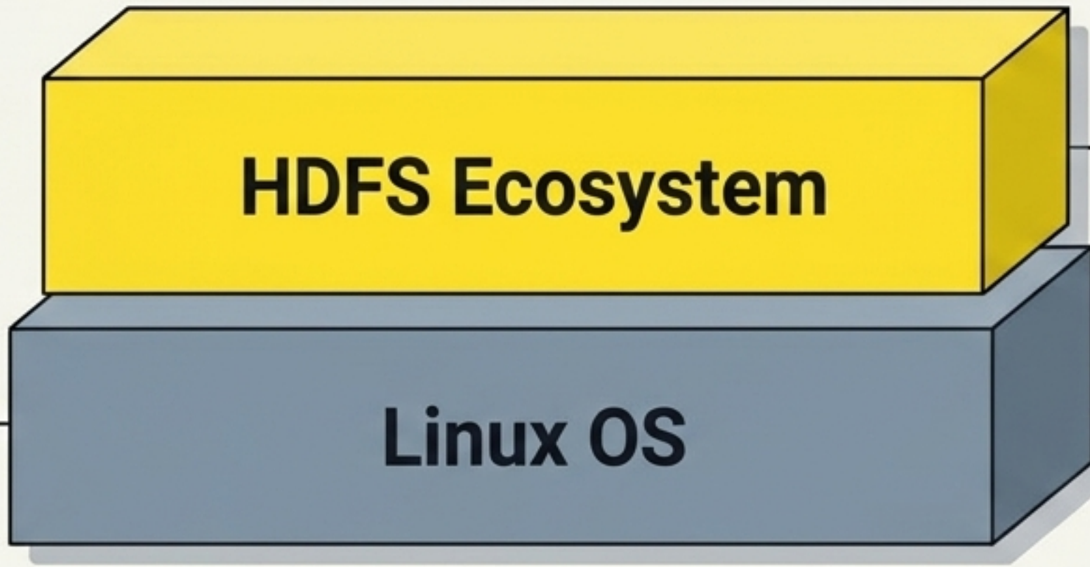
Verification

```
$ jps
```

Java Virtual Machine Process Status Tool.
If healthy, returns exactly five core services.

Critical: If these services stop in the task manager, the Hadoop application stops.

HDFS runs as a distributed application layer on top of Linux.



A distributed file system running on ports (e.g., `127.0.0.1`).

Uses local directories. Operations occur natively.

`Stateless Operations`

HDFS does not have a check-in/check-out concept.

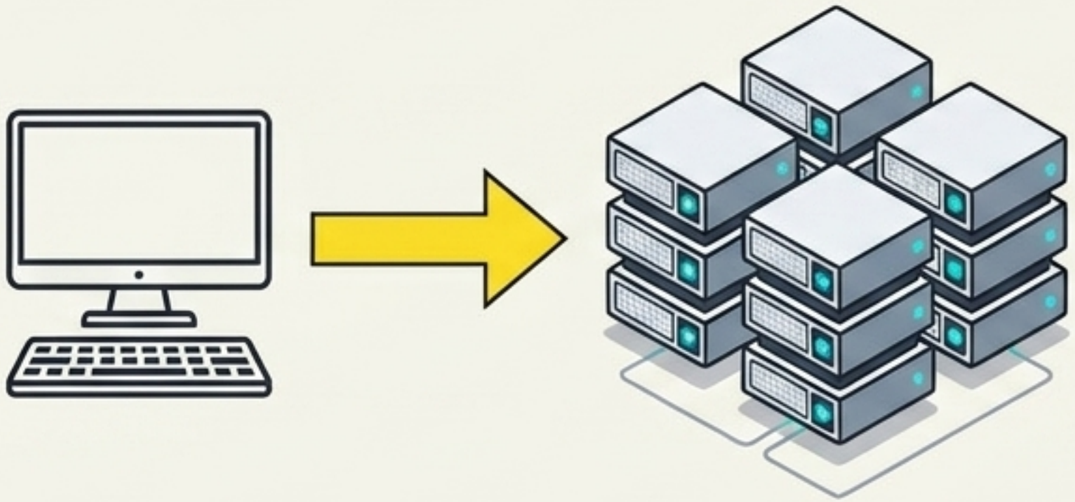
`No Persistent Directory`

You cannot use `cd` to navigate into an HDFS folder and stay there.

`Absolute Paths Required`

Every single HDFS command **must include the full, absolute path** to function. Standard Linux commands will not work inside the HDFS layer.

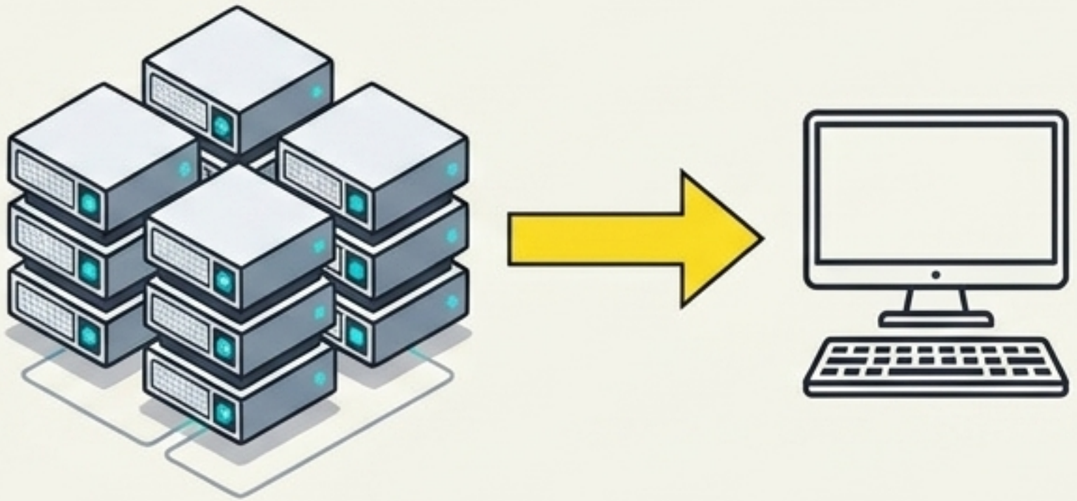
Data movement requires explicit directional commands.



Terminal

```
bin/hdfs dfs -put [linux_source] [hdfs_destination]
```

Mechanism: Copies a file from the local Linux operating system and inserts it into the specified HDFS directory.



Terminal

```
bin/hdfs dfs -get [hdfs_source] [linux_destination]
```

Mechanism: Fetches a distributed file from the HDFS cluster and reassembles it onto the local Linux filesystem.

Structural commands for internal HDFS management.

Create

```
bin/hdfs dfs -mkdir [path]
```

Creates a new directory.

Inspect

```
bin/hdfs dfs -ls [path]
```

Lists contents. (Add **-R** or use **-lsr** for recursive listing of all nested subdirectories).

Transfer

Copy: `bin/hdfs dfs -cp [source] [destination]`
Move: `bin/hdfs dfs -mv [source] [destination]`

Delete

File: `bin/hdfs dfs -rm [file]`
Directory: `bin/hdfs dfs -rm -r [directory]`

Analytics commands isolate file sizes and specific data blocks.

Data Reading

```
1 bin/hdfs dfs -cat [path]
```

Reads the entire file

```
2 cat [path] | head -n [X]
```

Returns only the top X lines

```
3 cat [path] | tail
```

Returns the bottom lines

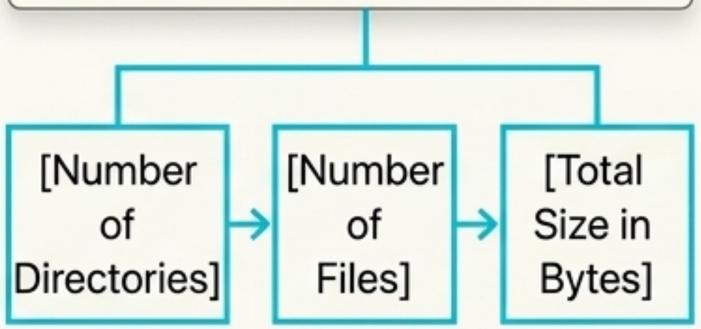
Size Verification

```
bin/hdfs dfs -du [path]
```

⚠ **Crucial Detail:** This command returns the file size strictly in Bytes</span (e.g., 84,854 bytes). It will never return sizes in KB, MB, or GB.

Directory Analytics

```
bin/hdfs dfs -count [path]
```



Returns these three specific metrics in exact order.

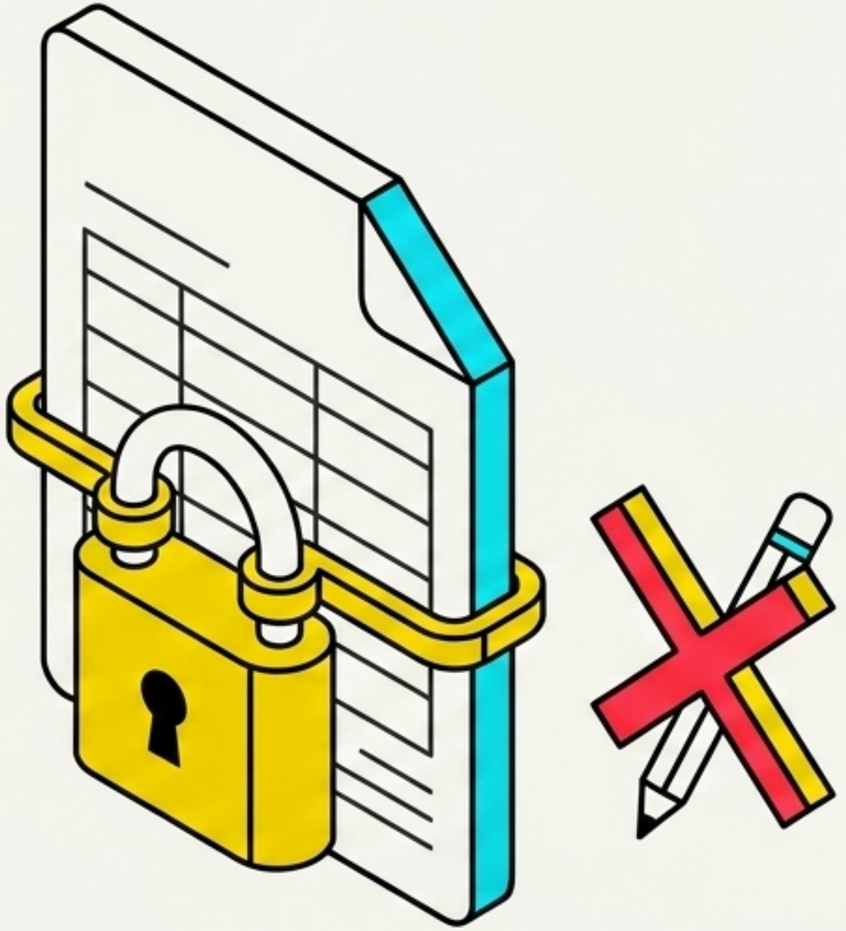
HDFS enforces strict data immutability

Zero Edits

Once a file is saved into HDFS, it cannot be modified, edited, or changed.

No Partial Retrieval

HDFS does not understand internal file logic. You cannot ask HDFS to natively fetch “only the top 10 rows” of a stored dataset.



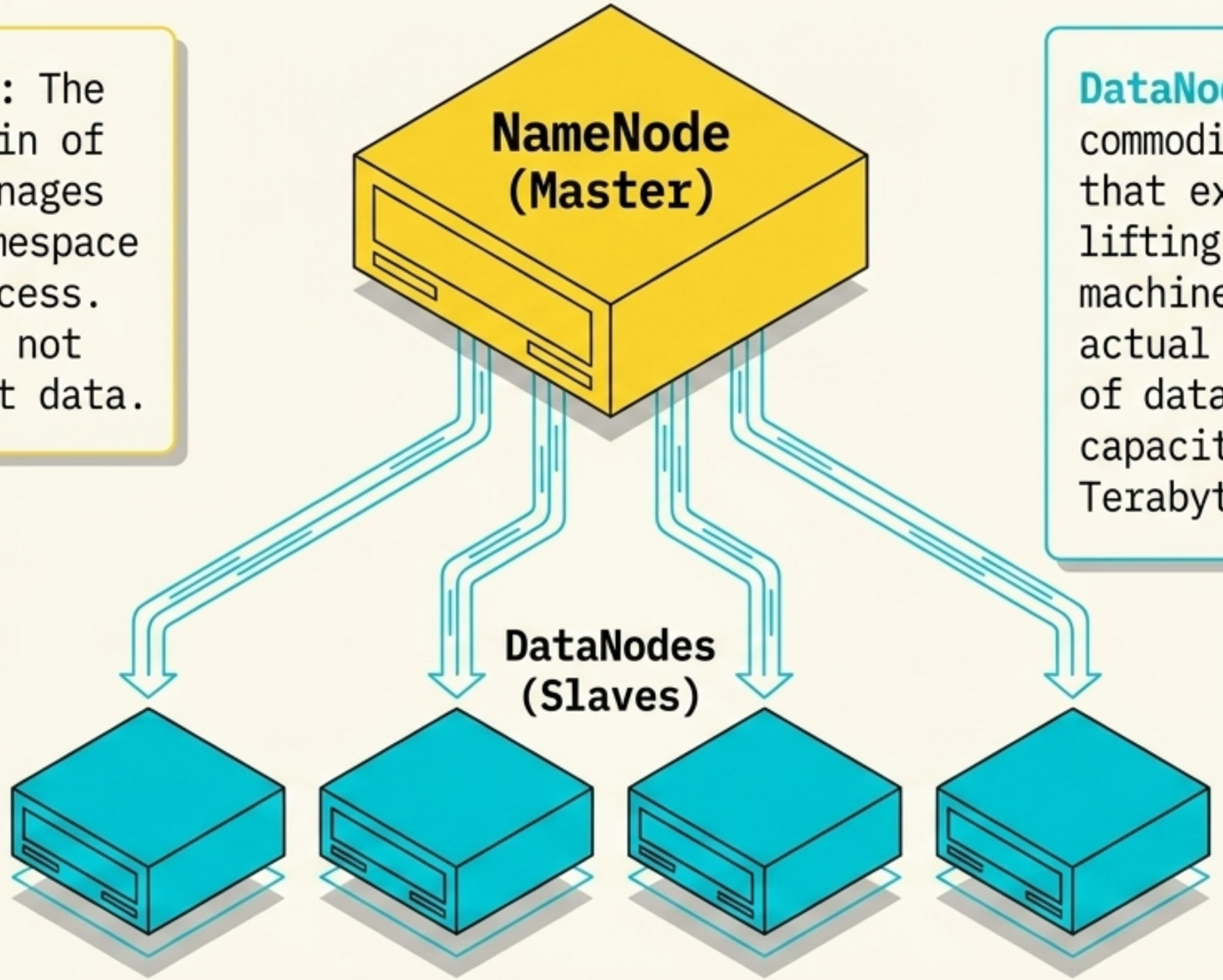
The Workflow

You store whole files, and you fetch whole files.

Any data manipulation, analysis, or selective querying requires external analytical tools sitting on top of HDFS (like Hive or Spark).

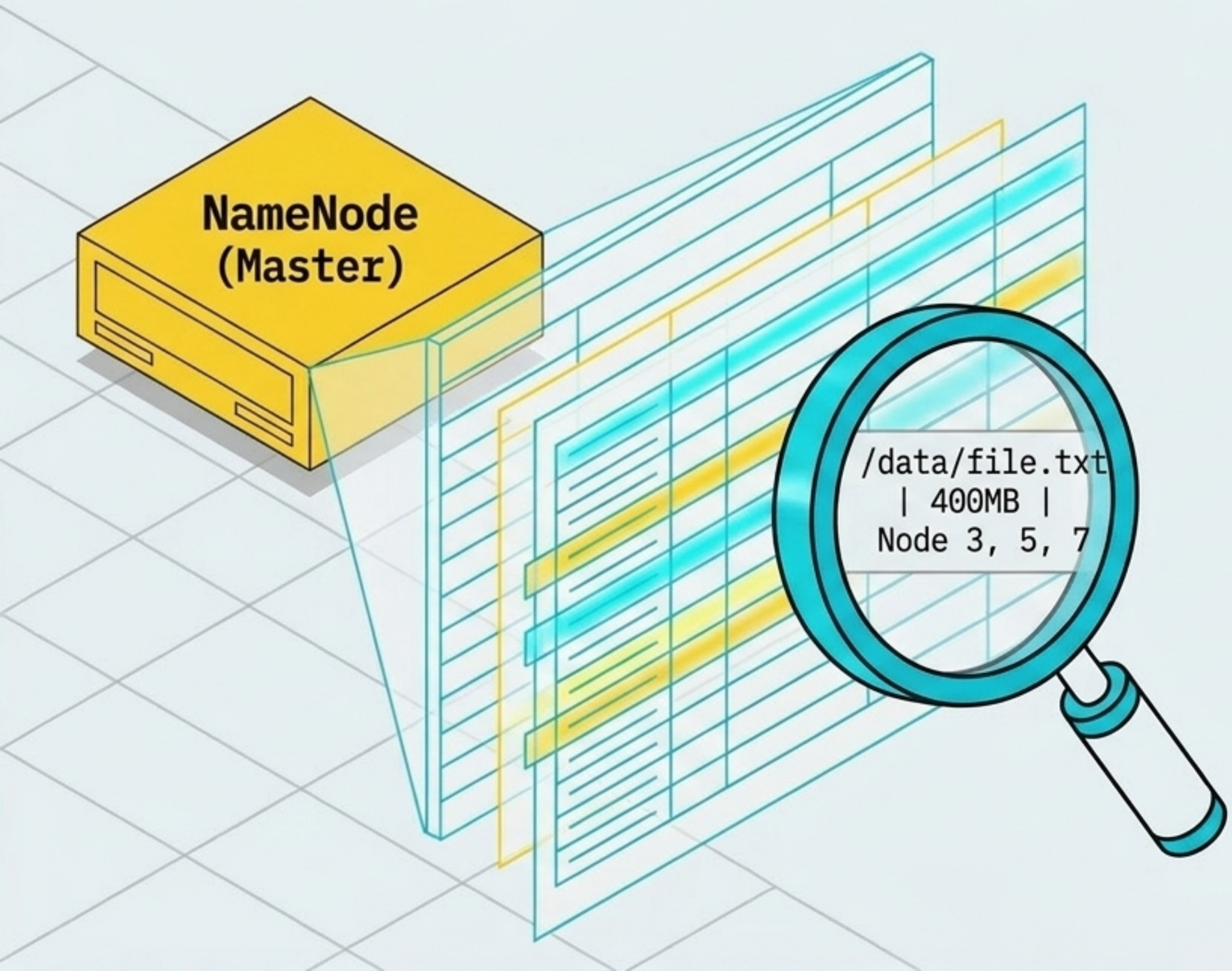
Production clusters operate on a Master/Slave architecture.

NameNode (Master): The administrative brain of the cluster. It manages the file system namespace and coordinates access. Crucially, it does not store actual client data.



DataNodes (Slaves): The commodity hardware servers that execute the heavy lifting. These physical machines store the actual chopped-up chunks of data. Their individual capacity is measured in Terabytes (TBs).

The **NameNode** relies entirely on its **Metadata ledger.**

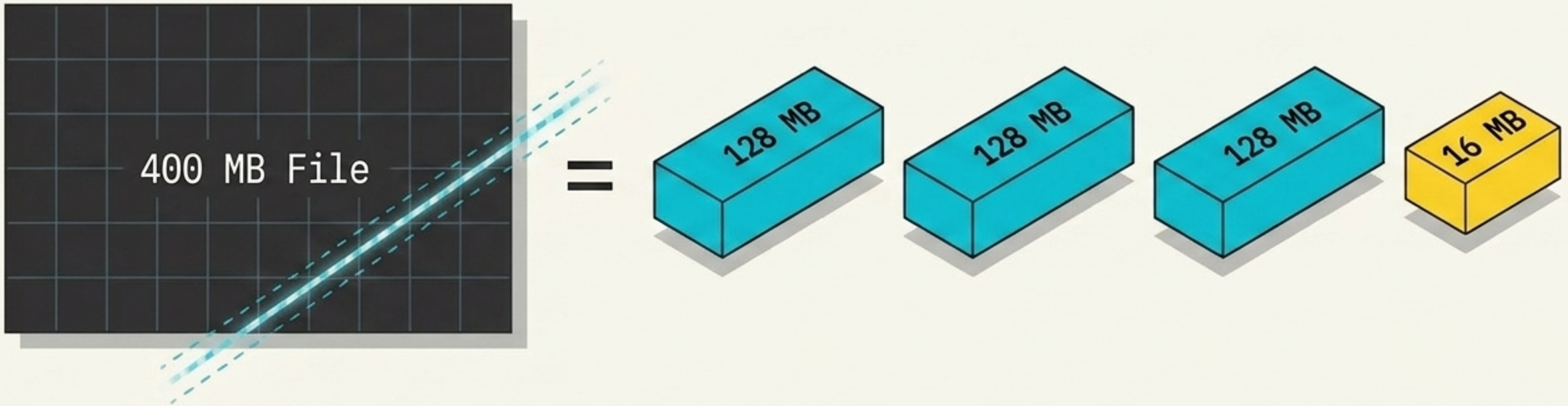


What does the NameNode actually know?

The Metadata is an index. For every single DataNode in the cluster (whether there are 10 or 1,000), the NameNode tracks:

1. Which specific files and directories exist.
2. File sizes and user permissions.
3. Exactly how much physical space is occupied vs. currently available on each individual DataNode.

Hadoop chops all data into uniform **128 MB** blocks.



The Rule (Hadoop 2.x):

The default block size for HDFS is strictly fixed at 128 MB (upgraded from 64 MB in Hadoop 1.x).

The Math (400 MB ÷ 128 MB):

When a client requests to store a 400 MB file, HDFS does not save it as a single entity. It mathematically divides it into 4 discrete blocks.

The Result:

The NameNode will distribute these 4 independent blocks across multiple available DataNodes to balance the system load.

Anatomy of a **–put** request execution.

1

Step 1 (Command)

Client executes `bin/hdfs dfs -put`

2

Step 2 (Request)

The request reaches the NameNode.

3

Step 3 (Lookup)

The NameNode scans its Metadata ledger to find DataNodes with available capacity.

4

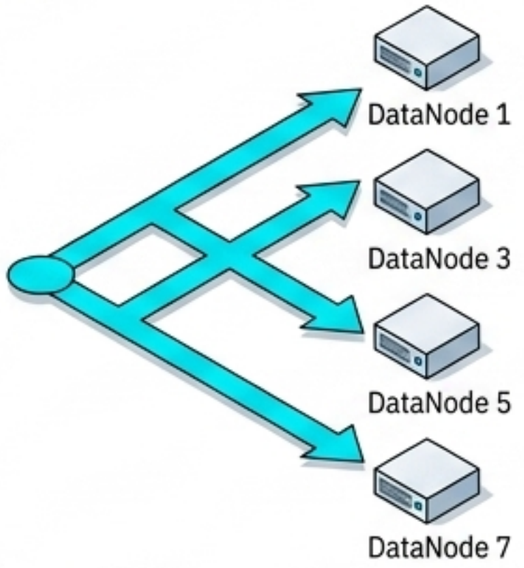
Step 4 (Response)

NameNode returns a specific list of target nodes (e.g., Nodes 1, 3, 5, and 7).

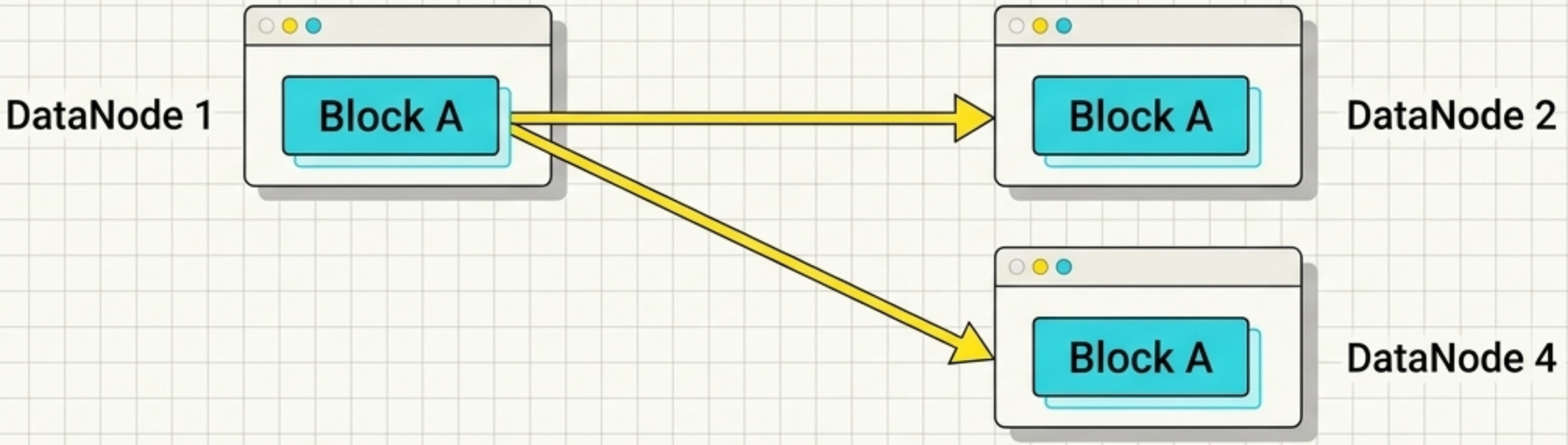
5

Step 5 (Parallel Execution)

The Client bypasses the NameNode and writes the 4 data blocks parallelly and directly to DataNodes 1, 3, 5, and 7.



Failsafe 1: Automatic 3x Data Replication



The Mechanism

By default, HDFS creates **3 identical replicas** of every single data block data block and scatters them across different DataNodes.

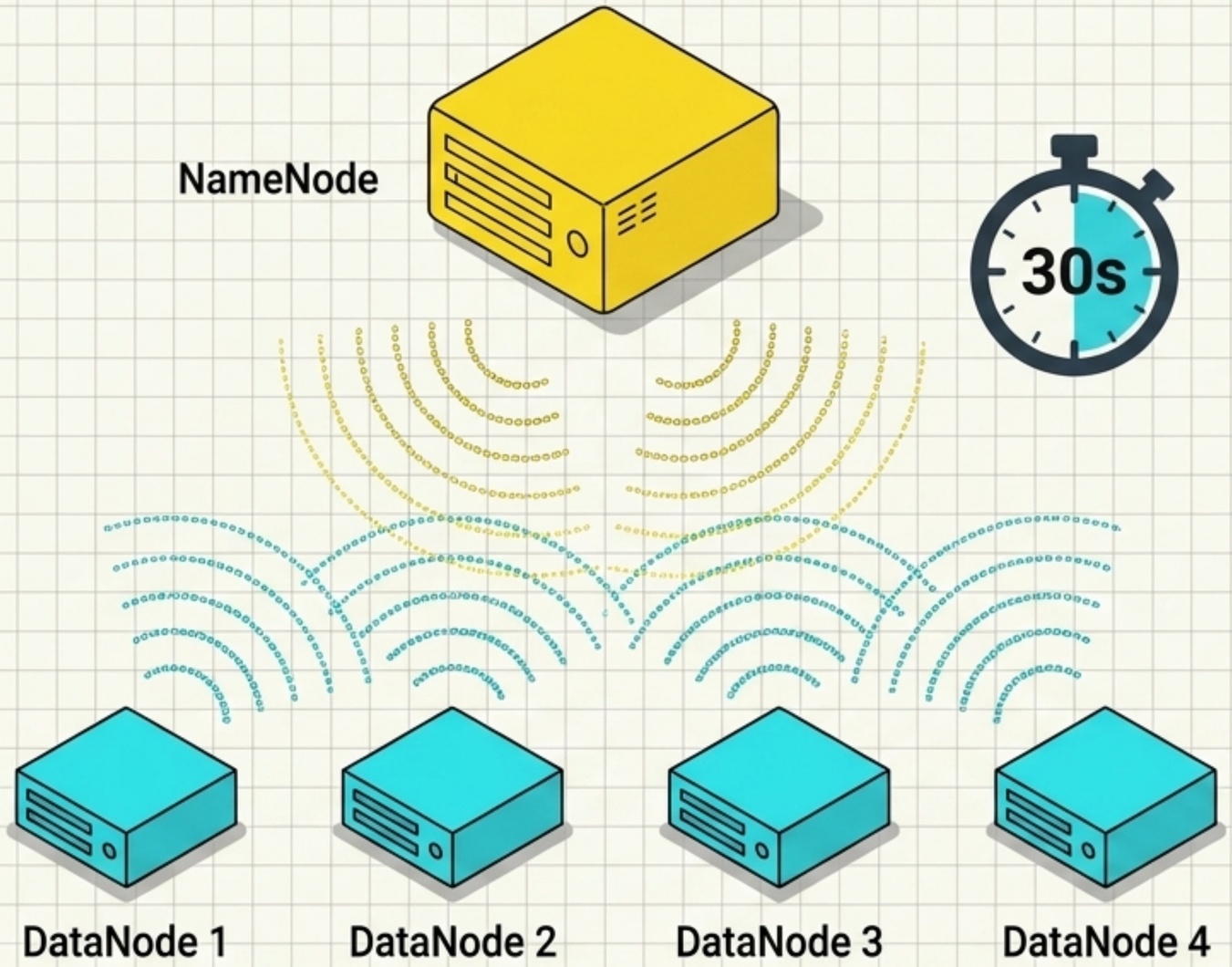
The Storage Cost

Storing a 400 MB file actually consumes `1.2 GB` of total cluster space (400 MB x 3).

The Recovery

If DataNode 1 suffers a catastrophic hardware crash, the client's `-get` request will not fail. The NameNode simply redirects the request to the exact replica waiting on DataNode 2.

Failsafe 2: Heartbeats and Block Reports.



The 30-Second Rule

Every 30 seconds, every active DataNode sends a 'Heartbeat' ping to the NameNode confirming 'I am alive.'

The Block Report

Accompanying the ping is a Block Report—a complete inventory of every data block currently stored on that specific server.

Failure Detection

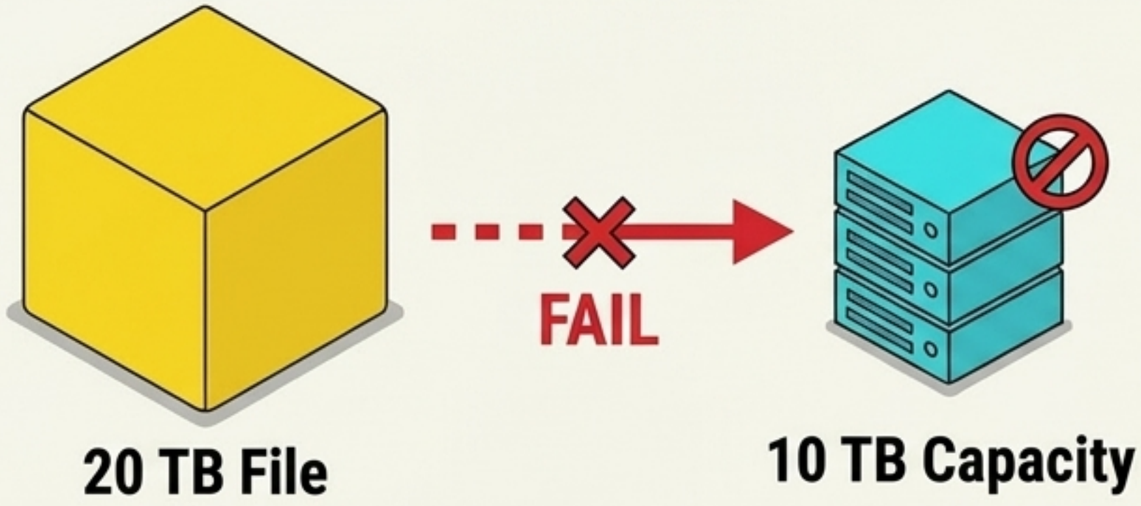
If the NameNode misses consecutive heartbeats from a DataNode, it formally declares the node dead, updates the Metadata, and reroutes all future traffic.

System resilience matrix.

	Event	Detection	Response
Slave Failure	An active DataNode crashes.	NameNode registers a missed 30-second heartbeat.	Client requests are instantly redirected to surviving nodes holding replicated blocks. Data remains safe and accessible.
Master Failure	The Active NameNode crashes.	Cluster loses its primary Metadata brain.	A configured 'Passive/Standby NameNode' immediately assumes the Active role. It regenerates the Metadata, ensuring the cluster remains operational without manual intervention.

Enterprise constraints: HDFS is bound by physical hardware.

Technical Warning



Physical Storage Limits: HDFS cannot defy physics. The `-put` command will outright fail if cluster capacity is exceeded. Scaling requires administration teams physically adding and configuring new server racks.

The Enterprise Reality

1 High Cost

Setting up physical on-premise clusters is extremely expensive and requires dedicated administration teams.

2 Specific Use Cases

HDFS is designed for massive datasets (GBs/TBs) requiring extreme security (like Banking data centers). It is highly inefficient and unnecessary for storing small, MB-sized files.